

## 第3章 数学

### 3.1 数论

在算法竞赛中，数论常常以各种面貌出现，但万变不离其宗。本节介绍几个最为常见的算法的例题及其代码实现。

**例3-1 【唯一分解】最小公倍数的最小和 (Minimum Sum LCM, UVa 10791)**

输入整数 $n$  ( $1 \leq n < 2^{31}$ )，求至少两个正整数，使得它们的最小公倍数为 $n$ ，且这些整数的和最小。输出最小的和。

**【代码实现】**

```

// 陈锋
#include <cmath>
#include <iostream>
using namespace std;
typedef long long LL;

LL solve(int n) {
    if (n == 1) return 2;
    int maxp = floor(sqrt(n) + 0.5), pf = 0; // 素因子(prime_factor)个数
    LL ans = 0;
    for (int i = 2; i < maxp; i++) {
        if (n % i) continue;
        int x = 1; // 新的素因子 i
        while (n % i == 0) n /= i, x *= i; // x = i^()
        ans += x, pf++;
    }
    if (n > 1) pf++, ans += n; // 注意, 此时 n 没被除尽, 也是一个素因子
    if (pf <= 1) ans++;
    return ans;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n, kase = 1; cin >> n && n; kase++)
        cout << "Case " << kase << ": " << solve(n) << "\n";
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 10-4
同类题目: Almost Prime Numbers, UVa 10539
          Perfect Pth Powers, UVa 10622
          Divisors, UVa 294
          A Research Problem, UVa 10837
*/

```

### 例 3-2 【模算术；幂取模】巨大的斐波那契数（Colossal Fibonacci Numbers, UVa 11582）

输入两个非负整数  $a$ ,  $b$  和正整数  $n$  ( $0 \leq a, b < 2^{64}$ ,  $1 \leq n \leq 1000$ )，你的任务是计算  $f(a^b)$  除以  $n$  的余数。其中， $f(0)=f(1)=1$ ，且对于所有非负整数  $i$ ， $f(i+2)=f(i+1)+f(i)$ 。

#### 【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const int NN = 1000 + 8;
typedef unsigned long long ULL;
int F[NN][NN * 6], period[NN];

int pow_mod(ULL a, ULL b, int MOD) {
    if (!b) return 1;
    int k = pow_mod(a, b / 2, MOD);
    k = k * k % MOD;
    if (b % 2) k = k * a % MOD;
    return k;
}

int solve(ULL a, ULL b, int MOD) {
    if (a == 0 || MOD == 1) return 0; // 需要注意边界情况
    int p = pow_mod(a % period[MOD], b, period[MOD]);
    return F[MOD][p];
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n = 2; n <= 1000; n++) {
        F[n][0] = 0, F[n][1] = 1;
        for (int i = 2; i++) {
            F[n][i] = (F[n][i - 1] + F[n][i - 2]) % n;
            if (F[n][i - 1] == 0 && F[n][i] == 1) {
                period[n] = i - 1;
                break;
            }
        }
    }
    ULL a, b;
    int T;
    cin >> T;
    for (int t = 0, n; t < T; t++)
        cin >> a >> b >> n, cout << solve(a, b, n) << endl;
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题10-1
同类题目: Raising Modulo Numbers, POJ 1995
*/

```

**例3-3 【模幂运算；素数判定】选择与除法（Choose and Divide, UVa 10375）**

已知  $C_m^n = m!/(n!(m-n)!)$ ，输入整数  $p, q, r, s$  ( $p \geq q, r \geq s, p, q, r, s \leq 10\,000$ )，计算  $C_p^q / C_r^s$ 。输出保证不超过  $10^8$ ，保留5位小数。

**【代码实现】**

```

// 刘汝佳
#include <cstdio>
#include <cmath>
#include <iostream>
#include <vector>
using namespace std;

const int NN = 10000;
vector<int> primes;
int E[NN];
// 乘以或除以 n, d=0 表示乘, d=-1 表示除
void add_integer(int n, int d) {
    for (size_t i = 0; n > 1 && i < primes.size(); i++) {
        while (n % primes[i] == 0)
            n /= primes[i], E[i] += d;
        // if (n == 1) break;           // 提前终止循环, 节约时间
    }
}

void add_factorial(int n, int d) {    // 阶乘中的每一个数字计入计算结果
    for (int i = 1; i <= n; i++) add_integer(i, d);
}

bool is_prime(int n) {
    int m = floor(sqrt(n) + 0.5);
    for (int a = 2; a <= m; a++)
        if (n % a == 0) return false;
    return true;
}

int main() {
    for (int i = 2; i <= NN; i++)
        if (is_prime(i)) primes.push_back(i);
    ios::sync_with_stdio(false), cin.tie(0);
    for (int p, q, r, s; cin >> p >> q >> r >> s;) {
        fill_n(E, NN, 0);
        add_factorial(p, 1), add_factorial(q, -1), add_factorial(p - q, -1);
        add_factorial(r, -1), add_factorial(s, 1), add_factorial(r - s, 1);
        double ans = 1;
        for (size_t i = 0; i < primes.size(); i++) ans *= pow(primes[i], E[i]);
        printf("%.5lf\n", ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 10-3
*/

```

### 例3-4 【数列求和】约瑟夫的数论问题 (Joseph' s Problem, NEERC 2005, UVa 1363)

输入正整数 $n$ 和 $k$  ( $1 \leq n, k \leq 10^9$ )，计算 $\sum_{i=1}^n k \bmod i$ 。

#### 【代码实现】

```
// 陈锋
#include <algorithm>
#include <cstdio>
using namespace std;
typedef long long LL;

int main() {
    for (int N, K; scanf("%d%d", &N, &K) == 2;) {
        LL ans = 0;
        if (N >= K) ans = (LL) (N - K) * K, N = K - 1;
        for (int i = 1, j; i <= N; i = j + 1) {
            j = min(K / (K / i), N);
            ans += (LL) (K % i + K % j) * (LL) (j - i + 1) / 2;
        }
        printf("%lld\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 10-25
*/
```

### 例3-5 【扩展欧几里得】整数对 (Pair of Integers, ACM/ICPC NEERC 2001, UVa 1654)

考虑一个不含前导0的正整数 $X$ ，把它去掉一个数字以后得到另外一个数 $Y$ 。输入 $X+Y$ 的值 $N$  ( $1 \leq N \leq 10^9$ )，输出所有可能的等式 $X+Y=N$ 。例如， $N=34$ 有两个解： $31+3=34$ ， $27+7=34$ 。

#### 【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
```

```

typedef long long LL;
void gcd(LL a, LL b, LL& d, LL& x, LL& y) {
    if (!b)
        d = a, x = 1, y = 0;
    else
        gcd(b, a % b, d, y, x), y -= x * (a / b);
}

const int MAXD = 10;
LL P10[MAXD];

inline int lenof(LL x) { return to_string(x).size(); }
void solve(LL N, map<LL, LL>& ans) {
    ans.clear();
    int D = lenof(N);
    if (N % P10[D - 1] == 0) ans[N] = 0;
    _for(i, 0, D) {
        _for(d, 0, 10) if (N > d * P10[i]) { // 解:  $10^i \cdot 11x + 2y = N - d \cdot 10^i$ 
            LL x, y, g, a = P10[i] * 11, b = 2, c = N - d * P10[i];
            gcd(a, b, g, x, y);
            if (c % g) continue;
            LL k = c / g;
            a /= g, b /= g, x *= k, y *= k; //  $x + kb, y - ka, 0 \leq x < 10^{(D-i-1)}, 0 \leq y < 10^i$ 
            for (LL step = 1 << 30; step >= 1; step >>= 1) {
                if (y >= step * a) y -= step * a, x += step * b;
                if (y < -step * a) y += step * a, x -= step * b;
            }
            while (y < 0) y += a, x -= b;
            if (y >= 0 && y < P10[i] && x >= 0 && x < P10[D - i - 1]) {
                LL X = x * P10[i + 1] + d * P10[i] + y, Y = x * P10[i] + y;
                if (X > Y && X + Y == N) ans[X] = Y;
            }
        }
    }
}

int main() {
    P10[0] = 1;
    _for(i, 1, MAXD) P10[i] = 10 * P10[i - 1];
    int T, N;
    scanf("%d", &T);
    map<LL, LL> ans;
    for (int t = 0; scanf("%d", &N) == 1; t++) {
        if (t) puts("");
        solve(N, ans);
        printf("%lu\n", ans.size());
        for (auto p : ans)
            printf("%lld + %0*lld = %d\n", p.first, lenof(p.first)-1, p.second, N);
    }
}

```



```

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 10-43
注意：解不定方程时的倍增逻辑
同类题目：青蛙的约会，POJ 1061
*/

```

### 例3-6 【素数筛法；阶乘的phi函数】帮帮Tomisu（Help Mr. Tomisu, UVa 11440）

给定正整数 $N$ 和 $M$ ，统计2和 $M!$ 之间有多少个整数 $x$ 满足： $x$ 的所有素因子都大于 $M$ （ $2 \leq N \leq 10^7$ ， $1 \leq M \leq N$ ， $N-M \leq 10^5$ ）。输出答案除以100 000 007的余数。例如， $N=100$ ， $M=10$ 时，答案为43 274 465。

#### 【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const LL MOD = 1e8 + 7;
const int MAXM = 1e7 + 4, MAXP = MAXM + 4;
LL PF[MAXM + 4]; // p[i]:phi(N!)
bool isPrime[MAXP];
vector<int> primes; // primes
void sieve() { // 素数筛法
    fill_n(isPrime, MAXP, true);
    _for(i, 2, MAXP) if (isPrime[i]) {
        LL j = i;
        for (j *= i; j < MAXP; j += i) isPrime[j] = false;
        primes.push_back(i);
    }
}

void init() { // 初始化  $\Phi(N!)$ 
    sieve();
    PF[1] = 1, PF[2] = 1;
    _rep(n, 3, MAXM) {
        if (isPrime[n])
            PF[n] = (n - 1) * PF[n - 1];
        else
            PF[n] = n * PF[n - 1];
        PF[n] %= MOD;
    }
}

int main() {
    init();
    for (LL N, M; scanf("%lld%lld", &N, &M) == 2 && N && M;) {
        LL ans = 1;
        _rep(i, M + 1, N) ans = (ans * i) % MOD;
        ans = (ans * PF[M] - 1) % MOD;
        printf("%lld\n", ans);
    }
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-26
同类题目：Semi-prime H-numbers, UVa 11105
*/

```

**例3-7 【gcd函数；phi函数】最大公约数之和一极限版II (GCD-Extreme(II), UVa 11426)**

输入正整数 $n$ ，求 $\gcd(1,2)+\gcd(1,3)+\gcd(2,3)+\cdots+\gcd(n-1,n)$ ，即所有满足 $1 \leq i < j \leq n$ 的数对 $(i,j)$ 所对应的 $\gcd(i,j)$ 之和。例如， $n=10$ 时答案为67， $n=100$ 时答案为13 015， $n=200\ 000$ 时答案为143 295 493 160。

**【代码实现】**

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const int NN = 4000000;
typedef long long LL;

int phi[NN + 1];
void phi_table(int n) {
    for (int i = 2; i <= n; i++) phi[i] = 0;
    phi[1] = 1;
    for (int i = 2; i <= n; i++)
        if (!phi[i])
            for (int j = i; j <= n; j += i) {
                if (!phi[j]) phi[j] = j;
                phi[j] = phi[j] / i * (i - 1);
            }
}
LL S[NN + 1], f[NN + 1];
int main() {
    phi_table(NN);
    fill_n(f, NN, 0); // 预处理 f
    for (int i = 1; i <= NN; i++)
        for (int n = i * 2; n <= NN; n += i) f[n] += i * phi[n / i];
    S[2] = f[2]; // 预处理 S
    for (int n = 3; n <= NN; n++) S[n] = S[n - 1] + f[n];
    for (int n; scanf("%d", &n) == 1 && n;) printf("%lld\n", S[n]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.2 节例题 9
同类题目：GCD Extreme (hard), SPOJ GCDEX2
*/

```

### 例 3-8 【欧拉 phi 函数预处理】交表 (Send a Table, UVa 10820)

有一道比赛题目：输入两个整数  $x, y$  ( $1 \leq x, y \leq n$ )，输出某个函数  $f(x, y)$ 。有位选手想交表（即事先计算出所有的  $f(x, y)$ ，写在源代码里），但是表太大了，源代码超过了比赛的限制，需要精简。

好在那道题目有一个性质，使得很容易根据 $f(x, y)$ 算出 $f(x*k, y*k)$ （其中 $k$ 是任意正整数），这样有一些 $f(x, y)$ 就不需要存在表里了。

输入 $n$  ( $n \leq 50\,000$ )，你的任务是统计最简的表里有多少个元素。例如， $n=2$ 时有3个元素，分别为 $(1,1)$ ,  $(1,2)$ ,  $(2,1)$ 。

### 【代码实现】

```
// 刘汝佳
#include <cmath>
#include <cstdio>
const int NN = 50000;
int Phi[NN + 1], PhiS[NN + 1];

void phi_table(int n) { // 预处理欧拉函数表
    Phi[1] = 0;
    for (int i = 2; i <= n; i++) {
        if (Phi[i]) continue;
        for (int j = i; j <= n; j += i) {
            int &pj = Phi[j];
            if (pj == 0) pj = j;
            pj = pj / i * (i - 1);
        }
    }
}

int main() {
    phi_table(NN);
    PhiS[0] = 0;
    for (int i = 1; i <= NN; i++) PhiS[i] = PhiS[i - 1] + Phi[i];
    for (int n; scanf("%d", &n) == 1 && n;) printf("%d\n", 2 * PhiS[n] + 1);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-7
同类题目：Farey Sequence, POJ 2578
*/
```

### 例3-9 【欧拉phi函数】树林里的树 (Trees in a Wood, UVa 10214)

在满足 $|x| \leq a$ ,  $|y| \leq b$  ( $a \leq 2000$ ,  $b \leq 2\,000\,000$ ) 的网格中，除原点外的整点（即 $x, y$ 坐标均为整数的点）各种着一棵树。树

的半径可以忽略不计，但是可以相互遮挡。求从原点能看到多少棵树。设这个值为 $K$ ，要求输出 $K/N$ ，其中 $N$ 为网格中树的总数。如图3-1所示，只有黑色的树可见。

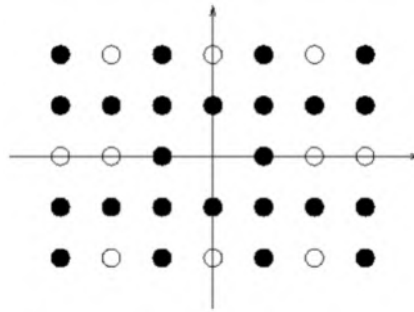


图3-1 树林里的树

### 【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
const int MAXA = 2000 + 4;
LL Phi[MAXA];
void init() {
    fill_n(Phi, MAXA, 0);
    Phi[1] = 1;
    _for(i, 2, MAXA) if (Phi[i] == 0) {
        for (LL j = i; j < MAXA; j += i) {
            LL& pj = Phi[j];
            if (pj == 0) pj = j;
            pj = pj / i * (i - 1);
        }
    }
}
int main() {
    init();
    for (int A, B; scanf("%d%d", &A, &B) == 2 && A && B; ) {
        LL P = 0;
        for (int x = 1; x <= A; x++) {
            int k = B / x;
            P += k * Phi[x];
            for (int y = k * x + 1; y <= B; y++) if (gcd(x, y) == 1) P++;
        }
        double ans = 4 * (P + 1);
        LL N = 4LL * A * B + 2LL * A + 2LL * B;
        printf("%.7lf\n", ans / N);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-27
同类题目：Primitive Roots, POJ 1284
*/

```

### 例3-10 【中国剩余定理】数论难题（Code Feat, UVa 11754）

有一个正整数 $N$ 满足 $C$ 个条件，每个条件都形如“它除以 $X$ 的余数在集合 $\{Y_1, Y_2, \dots, Y_k\}$ 中”，所有条件中的 $X$ 两两互素，你的任务是找出最小的 $S$ 个解。

#### 【代码实现】

```

// 刘汝佳
typedef long long LL;

// 即使 a, b 在 int 范围内, x 和 y 有可能超出 int 范围
void gcd(LL a, LL b, LL& d, LL& x, LL& y) {
    if (!b) d = a, x = 1, y = 0;
    else gcd(b, a % b, d, y, x), y -= x * (a / b);
}

// n 个方程: x=a[i](mod m[i]) (0<=i<n)
LL china(int n, int* a, int* m) {
    LL M = 1, d, y, x = 0;
    for (int i = 0; i < n; i++) M *= m[i];
    for (int i = 0; i < n; i++) {
        LL w = M / m[i];
        gcd(m[i], w, d, d, y);
        x = (x + y * w * a[i]) % M;
    }
    return (x + M) % M;
}

#include <algorithm>
#include <cstdio>
#include <set>
#include <vector>
using namespace std;

const int maxc = 9, maxk = 100, LIMIT = 10000;
set<int> values[maxc];
int C, X[maxc], k[maxc], Y[maxc][maxk];

void solve_enum(int S, int bc) {
    for (int c = 0; c < C; c++)
        if (c != bc) {
            values[c].clear();
            for (int i = 0; i < k[c]; i++) values[c].insert(Y[c][i]);
        }
    for (int t = 0; S != 0; t++) {
        for (int i = 0; i < k[bc]; i++) {
            LL n = (LL)X[bc] * t + Y[bc][i];
            if (n == 0) continue; // 只输出正数解
            bool ok = true;
            for (int c = 0; c < C; c++)
                if (c != bc)
                    if (!values[c].count(n % X[c])) {
                        ok = false;
                        break;
                    }
        }
    }
}

```



```

        if (ok) {
            printf("%lld\n", n);
            if (--S == 0) break;
        }
    }
}

int a[maxc]; // 搜索对象，用于中国剩余定理（孙子定理）
vector<LL> sol;

void dfs(int dep) {
    if (dep == C)
        sol.push_back(china(C, a, X));
    else
        for (int i = 0; i < k[dep]; i++) a[dep] = Y[dep][i], dfs(dep + 1);
}

void solve_china(int S) {
    sol.clear();
    dfs(0);
    sort(sol.begin(), sol.end());

    LL M = 1;
    for (int i = 0; i < C; i++) M *= X[i];

    vector<LL> ans;
    for (int i = 0; S != 0; i++) {
        for (int j = 0; j < sol.size(); j++) {
            LL n = M * i + sol[j];
            if (n > 0) {
                printf("%lld\n", n);
                if (--S == 0) break;
            }
        }
    }
}

int main() {
    int S;
    while (scanf("%d%d", &C, &S) == 2 && C) {
        LL tot = 1;
        int bestc = 0;
        for (int c = 0; c < C; c++) {
            scanf("%d%d", &X[c], &k[c]);
            tot *= k[c];
            for (int i = 0; i < k[c]; i++) scanf("%d", &Y[c][i]);
            sort(Y[c], Y[c] + k[c]);
            if (k[c] * X[bestc] < k[bestc] * X[c]) bestc = c;
        }
    }
}

```

```

    }
    if (tot > LIMIT)
        solve_enum(S, bestc);
    else
        solve_china(S);
    printf("\n");
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.2 节例题 10
同类题目：Strange Way to Express Integers, POJ 2891
*/

```

### 例3-11 【Hash；线性筛；素数分解】可怕的诗篇（A Horrible Poem, POI 2012，牛客NC 50316）

给出一个由小写英文字母组成的字符串 $S$ ，再给出 $q$ 个询问，要求找出某个子串 $S[a,b]$ 的最短循环节。如果字符串 $B$ 是字符串 $A$ 的循环节，那么 $A$ 可以由 $B$ 重复若干次得到（ $1 \leq a \leq b \leq n \leq 5 \times 10^5$ ， $q \leq 2 \times 10^6$ ）。

#### 【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;

const int NN = 5e5 + 4, x = 263;
typedef unsigned long long ULL;
typedef long long LL;

ULL XP[NN];
void initXP() {
    XP[0] = 1;
    for (size_t i = 1; i < NN; i++) XP[i] = x * XP[i - 1];
}
template <size_t SZ>
struct StrHash {
    size_t N;
    ULL H[SZ];

    void init(const char* pc, size_t n = 0) {
        if (XP[0] != 1) initXP();
        if (n == 0) n = strlen(pc);
        N = n;
        assert(N > 0);
        assert(N + 1 < SZ);
        H[N] = 0;
        for (int i = N - 1; i >= 0; --i) H[i] = pc[i] - 'a' + 1 + x * (H[i + 1]);
    }
}

```

```

void init(const string& S) { init(S.c_str(), S.size()); }
inline ULL hash(size_t i, size_t j) { // hash[i, j]
    // assert(i <= j);
    // assert(j < N);
    return H[i] - H[j + 1] * XP[j - i + 1];
}
inline ULL hash() { return H[0]; }
};

StrHash<NN> hs;
char S[NN];
int lastP[NN], primes[NN], pCnt;
void sieve(int N) {
    pCnt = 0;
    fill_n(lastP, N, 0);
    int* P = primes;
    for (int i = 2; i < N; ++i) {
        int& l = lastP[i]; // i 的最小素因子
        if (l == 0) l = i, P[pCnt++] = i; // i 是素数
        for (int j = 0; j < pCnt && P[j] <= l && P[j] * i < N; ++j)
            lastP[i * P[j]] = P[j]; // i*p 的最小素因子是 p
    }
}

int find_rep(int a, int b) {
    int L = b - a + 1, xl = L;
    while (xl > 1) {
        int p = lastP[xl]; // 尝试每一个素因子
        if (hs.hash(a, b - L / p) == hs.hash(a + L / p, b)) L /= p;
        xl /= p;
    }
    return L;
}

int main() {
    int n, q;
    S[0] = '|';
    scanf("%d%s%d", &n, S + 1, &q);
    hs.init(S, n + 1), sieve(n + 1);
    for (int i = 0, a, b; i < q; i++)
        scanf("%d%d", &a, &b), printf("%d\n", find_rep(a, b));
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.3 节例题 12
注意：本题是如何利用线性筛的性质来对任意数字做唯一分解的
同类题目：The Embarrassed Cryptographer, POJ 2635
*/

```

**例3-12 【莫比乌斯反演】可见格点 (Visible Lattice Points, Indian ICPC training camp, SPOJ VLATTICE)**

对于三维坐标系中的点 $\{(x, y, z) \mid 0 \leq x, y, z \leq N\}$ ,  $1 \leq N \leq 10^6$ ,  
问: 这些点中有哪些是原点可以看到的 (原点除外) ?

**【代码实现】**

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)
typedef long long LL;
const int MAXN = 1000000 + 4;
valarray<bool> isPrime(true, MAXN);
valarray<LL> Mu(0LL, MAXN), Lp(0LL, MAXN);
vector<LL> Ps;
void sieve(int N) {
    Ps.clear(), Mu[1] = 1;
    _for(i, 2, N) {
        LL& l = Lp[i];
        if (l == 0) Ps.push_back(i), Mu[i] = -1, l = i;
        for (size_t j = 0; j < Ps.size() && Ps[j] <= l && Ps[j] * i < N; ++j) {
            LL p = Ps[j];
            Lp[i * p] = p;
            if (i % p == 0) {
                Mu[i * p] = 0;
                break;
            }
            Mu[i * p] = -Mu[i];
        }
    }
}

int main() {
    sieve(MAXN);
    int T, N;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &N);
        LL ans = 3; // 3个坐标轴上的点
        _rep(d, 1, N) {
            LL k = N / d;
            ans += Mu[d] * k * k * (k + 3); // (x, y, z)个数以及3个平面上的(x, y)个数
        }
        printf("%lld\n", ans);
    }
    return 0;
}

/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.5 节例题 13
同类题目：GCD, HDU 1695
*/

```

**例3-13 【莫比乌斯反演；GCD】墨菲斯（Mophues, ACM/ICPC 杭州在线2013, HDU 4746）**

给出  $Q$  ( $1 \leq Q \leq 5000$ ) 个询问，每个询问给出3个整数  $N, M, P$  ( $N, M, P \leq 5 \times 10^5$ )，求  $\sum_{i=1}^N \sum_{j=1}^M [h(\gcd(i, j)) \leq P]$ 。其中， $h(x)$ 表示 $x$ 的唯一分解中的素因子个数，如 $12=2^2 \times 3$ ， $h(12)=3$ 。注意，这里 $h(1)=0$ 。

**【代码实现】**

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

const int MAXN = 500000 + 4, MAXP = MAXN;
vector<bool> isPrime(MAXP, true);
vector<LL> Mu(MAXP), Primes, H(MAXP, 1);
LL G[MAXN][20];
void sieve() { // 筛法计算素数以及mu函数
    Mu[1] = 1, H[1] = 0;
    for (int i = 2; i < MAXP; ++i) {
        if (isPrime[i]) Primes.push_back(i), Mu[i] = -1, H[i] = 1;
        for (size_t j = 0; j < Primes.size(); ++j) {
            LL p = Primes[j], t = p * i;
            if (t >= MAXP) break;
            isPrime[t] = false, H[t] = H[i] + 1;
            if (i % p == 0) {
                Mu[t] = 0;
                break;
            }
            Mu[t] = -Mu[i];
        }
    }
    memset(G, 0, sizeof(G));
    for (int n = 1; n < MAXN; n++) {
        for (int k = 1, T = n; T < MAXN; ++k, T += n)
            G[T][H[n]] += Mu[k]; //  $\sum \mu(T/n) |h(n)=P$ 
    }

    for (int n = 1; n < MAXN; n++)
        for (int p = 1; p < 20; p++)
            G[n][p] += G[n][p - 1]; //  $\sum \mu(T/n) |h(n) \leq P$ 
    for (int n = 1; n < MAXN; n++)
        for (int p = 0; p < 20; p++) G[n][p] += G[n - 1][p]; //  $\sum \mu(T/n) |h(n) \leq P$ 
}

LL N, M, P;
LL solve() {
    if (P >= 20) return N * M;
    if (N > M) swap(N, M);
    LL ans = 0;
    for (int T = 1, et = 0; T <= N; T = et + 1) {
        et = min(N / (N / T), M / (M / T)); //  $t \in [T, et], N/t, M/t$  相同
        ans += (G[et][P] - G[T - 1][P]) * (N / T) * (M / T);
    }
    return ans;
}

int main() {
    sieve();
    int Q;
    scanf("%d", &Q);
    while (Q--) {
        scanf("%lld%lld%lld", &N, &M, &P);
        printf("%lld\n", solve());
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.5 节例题 14
同类题目：Count a*b, ACM/ICPC 长春 2015, HDU 5528
*/

```



### 例 3-14 【高次模方程；BSGS；原根】信息解密（Decrypt Messages, 上海2009, UVa 1457）

假设从2000年1月1日00:00:00到现在经过了 $x$ 秒，计算 $x^q \bmod p$ ，设答案为 $a$ 。这里， $p$ 严格大于 $x$ 。已知 $p, q, a$ （ $2 < p \leq 1\,000\,000\,007$ ， $1 < q \leq 10$ ， $0 \leq a < p$ ， $p$ 保证为素数），求现在时刻的所有可能值。

**提示：**如果一个年份是4的倍数但不是100的倍数，或者这个年份是400的倍数，则这个年份是闰年。闰年的二月有29天，其他年（平年）的二月只有28天。在本题中，如果年份除以10的余数为5或者8，则这一年的最后还会有一个“闰秒”。比如，2005年12月31日23:59:59的下一秒是2005年12月31日23:59:60，再下一秒才是2006年1月1日00:00:00。

#### 【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <map>
#include <algorithm>
```

```

#include <iostream>
using namespace std;

typedef long long LL;
// 日期时间部分
const int SECONDS_PER_DAY = 24 * 60 * 60;
const int num_days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
bool is_leap(int year) {
    if (year % 400 == 0) return true;
    if (year % 4 == 0) return year % 100 != 0;
    return false;
}

int leap_second(int year, int month) {
    return ((year % 10 == 5 || year % 10 == 8) && month == 12) ? 1 : 0;
}

void print(int year, int month, int day, int hh, int mm, int ss) {
    printf("%d.%02d.%02d %02d:%02d:%02d\n", year, month, day, hh, mm, ss);
}

void print_time(LL t) {
    int year = 2000;
    while(1) {
        int days = is_leap(year) ? 366 : 365;
        LL sec = (LL)days * SECONDS_PER_DAY + leap_second(year, 12);
        if(t < sec) break;
        t -= sec;
        year++;
    }

    int month = 1;
    while(1) {
        int days = num_days[month-1];
        if(is_leap(year) && month == 2) days++;
        LL sec = (LL)days * SECONDS_PER_DAY + leap_second(year, month);
        if(t < sec) break;
        t -= sec;
        month++;
    }

    if(leap_second(year, month) && t == 31 * SECONDS_PER_DAY)
        print(year, 12, 31, 23, 59, 60);
    else {
        int day = t / SECONDS_PER_DAY + 1;
        t %= SECONDS_PER_DAY;
        int hh = t / (60*60);
        t %= 60*60;
        int mm = t / 60;
    }
}

```

```

        t %= 60;
        int ss = t;
        print(year, month, day, hh, mm, ss);
    }
}

// 数论部分

LL gcd(LL a, LL b) {
    return b ? gcd(b, a%b) : a;
}

// 求 d = gcd(a, b), 以及满足 ax+by=d 的 (x,y) (注意, x 和 y 可能为负数)
// 扩展 euclid 算法
void gcd(LL a, LL b, LL& d, LL& x, LL& y) {
    if(!b){ d = a; x = 1; y = 0; }
    else{ gcd(b, a%b, d, y, x); y -= x*(a/b); }
}

// 注意, 返回值可能是负的
int pow_mod(LL a, LL p, int MOD) {
    if(p == 0) return 1;
    LL ans = pow_mod(a, p/2, MOD);
    ans = ans * ans % MOD;
    if(p%2) ans = ans * a % MOD;
    return ans;
}

// 注意, 返回值可能是负的
int mul_mod(LL a, LL b, int MOD) {
    return a * b % MOD;
}

// 求 ax = 1 (mod MOD) 的解, 其中 a 和 MOD 互素
// 注意, 由于 MOD 不一定为素数, 因此不能直接用 pow_mod(a, MOD-2, MOD) 求解
// 解法: 求出 ax + MODy = 1 的解 (x,y), 则 x 即为所求
int inv(LL a, int MOD) {
    LL d, x, y;
    gcd(a, MOD, d, x, y);
    return (x + MOD) % MOD; // 这里的 x 可能是负数, 因此要调整
}

// 解模方程 (即离散对数) a^x = b, 要求 MOD 为素数
// 解法: Shank 的大步小步算法
int log_mod(int a, int b, int MOD) {
    int m, v, e = 1, i;
    m = (int)sqrt(MOD);
    v = inv(pow_mod(a, m, MOD), MOD);
    map<int,int> x;

```

```

x[1] = 0;
for(i = 1; i < m; i++){ e = mul_mod(e, a, MOD); if (!x.count(e)) x[e] = i; }
for(i = 0; i < m; i++){
    if(x.count(b)) return i*m + x[b];
    b = mul_mod(b, v, MOD);
}
return -1;
}

```

// 返回 MOD (不一定是素数) 的某一个原根, phi 为 MOD 的欧拉函数值 (若 MOD 为素数则  $\phi = \text{MOD} - 1$ )  
 // 解法: 考虑  $\phi(\text{MOD})$  的所有素因子 p, 如果所有  $m^{(\phi/p)} \bmod \text{MOD}$  都不等于 1, 则 m 是 MOD 的原根

```

int get_primitive_root(int MOD, int phi) {
    // 计算 phi 的所有素因子
    vector<int> factors;
    int n = phi;
    for(int i = 2; i*i <= n; i++) {
        if(n % i != 0) continue;
        factors.push_back(i);
        while(n % i == 0) n /= i;
    }
    if(n > 1) factors.push_back(n);

    while(1) {
        int m = rand() % (MOD-2) + 2; // m = 2~MOD-1
        bool ok = true;
        for(int i = 0; i < factors.size(); i++)
            if(pow_mod(m, phi/factors[i], MOD) == 1) { ok = false; break; }
        if(ok) return m;
    }
}

```

// 解线性模方程  $ax = b \pmod{n}$ , 返回所有解 (模 n 剩余系)

// 解法: 令  $d = \gcd(a, n)$ , 两边同时除以 d 后得  $a'x = b' \pmod{n'}$ , 由于此时  $\gcd(a', n') = 1$ , 两边同时左乘  $a'$  在模  $n'$  中的逆即可, 最后把模  $n'$  剩余系中的解转化为模 n 剩余系

```

vector<LL> solve_linear_modular_equation(int a, int b, int n) {
    vector<LL> ans;
    int d = gcd(a, n);
    if(b % d != 0) return ans;
    a /= d; b /= d;
    int n2 = n / d;
    int p = mul_mod(inv(a, n2), b, n2);
    for(int i = 0; i < d; i++)
        ans.push_back(((LL)i * n2 + p) % n);
    return ans;
}

```

// 解高次模方程  $x^q = a \pmod{p}$ , 返回所有解 (模 n 剩余系)

// 解法: 设 m 为 p 的一个原根, 且  $x = m^y$ ,  $a = m^z$ , 则  $m^{qy} = m^z \pmod{p}$ , 因此  $qy = z \pmod{p-1}$ , 解线性模方程即可

```

vector<LL> mod_root(int a, int q, int p) {
    vector<LL> ans;
    if(a == 0) {
        ans.push_back(0);
        return ans;
    }
    int m = get_primitive_root(p, p-1); // p是素数, 因此phi(p)=p-1
    int z = log_mod(m, a, p);
    ans = solve_linear_modular_equation(q, z, p-1);
    for(int i = 0; i < ans.size(); i++)
        ans[i] = pow_mod(m, ans[i], p);
    sort(ans.begin(), ans.end());
    return ans;
}

int main() {
    int T, P, Q, A;
    cin >> T;
    for(int kase = 1; kase <= T; kase++) {
        cin >> P >> Q >> A;
        vector<LL> ans = mod_root(A, Q, P);
        cout << "Case #" << kase << ":" << endl;
        if (ans.empty()) {
            cout << "Transmission error" << endl;
        } else {
            for(int i = 0; i < ans.size(); i++) print_time(ans[i]);
        }
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.5.1 节例题 22
*/

```

### 例3-15 【扩展BSGS；离散对数】幂取模反转（Power Modulo Inverted, SPOJ MOD）

给定 $a$ ,  $b$ 和 $p$  ( $1 \leq a, b, p \leq 10^9$ )，求出最小的 $x$ ，使其满足 $a^x \equiv b \pmod{p}$ 。

#### 【代码实现】

```
// 陈锋
#include <cassert>
#include <cstring>
#include <cmath>
#include <ctime>
#include <iostream>
#include <map>
#include <vector>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
```

```

#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;

LL gcd(LL a, LL b) { return b == 0 ? a : gcd(b, a % b); }
void gcd(LL a, LL b, LL& d, LL& x, LL& y) {
    if (b == 0)
        d = a, x = 1, y = 0;
    else
        gcd(b, a % b, d, y, x), y -= x * (a / b);
}
LL mul_mod(LL a, LL b, LL n) { return a * b % n; };
LL pow_mod(LL a, LL p, LL n) {
    if (p == 0) return 1;
    LL ans = pow_mod(a, p / 2, n);
    ans = mul_mod(ans, ans, n);
    if (p % 2 == 1) ans = mul_mod(ans, a, n);
    return ans;
}
LL inv(LL a, LL n) {
    LL d, x, y;
    gcd(a, n, d, x, y);
    return d == 1 ? (x + n) % n : -1;
}

// 求出最小的 x, 使之满足  $c \cdot a^x \equiv b \pmod p$ , 此时  $\gcd(a, p) = 1$ 
LL solve(const LL a, const LL c, LL b, const LL p) {
    LL m = (LL)ceil(sqrt(p)), e = c;
    map<LL, int> indice;
    indice[e] = 0;
    _for(i, 1, m) {
        e = mul_mod(e, a, p);
        if (!indice.count(e)) indice[e] = i;
    }
    LL iA = inv(pow_mod(a, m, p), p); //  $A^{-1}$ 
    _for(i, 0, m) { // 判断所有的  $a^{(jm+k)}$ ,  $k = [0, m)$ 
        if (indice.count(b)) return i * m + indice[b];
        b = mul_mod(b, iA, p); // 求出 indice 中是否存在  $(a^{jm})^{-1} \cdot b$ 
    }

    return -1;
}

bool simplify(const LL& a, LL& b, LL& p, LL& c, LL& cnt) {
    c = 1, cnt = 0;
    while (true) {
        LL d = gcd(a, p);
        if (d == 1) return true;
        if (b % d != 0) return false;
        b /= d, p /= d, c = mul_mod(c, a / d, p), cnt++;
    }
}

```

```

    }
    return false;
}

int main() {
    LL a, b, p, ans; //  $a^x \equiv b \pmod{p}$ , 求最小的 x
    while (cin >> a >> p >> b && a && b && p) {
        a %= p, b %= p, ans = -1;
        LL c, cnt, e = 1;
        _for(i, 0, 50) {
            if (e == b) {
                ans = i;
                break;
            }
            e = mul_mod(e, a, p);
        }

        if (ans == -1 && simplify(a, b, p, c, cnt)) ans = solve(a, c, b, p) + cnt;
        if (ans == -1)
            cout << "No Solution" << endl;
        else
            cout << ans << endl;
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 2.3.2 节例题 11
同类题目: UVa 11916, Emoogles Grid
          随机数生成器, SDOI 2013, NC 20362
*/

```

## 本节例题列表

本节讲解的例题及其囊括的知识点，如表3-1所示。

表3-1 数论例题归纳